



دانشگاه صنعتی امیرکبیر

(پلی تکنیک تهران)

دانشکده علوم کامپیوتر

پایان نامه کارشناسی ارشد

پروژه نرم افزار ریاضی

نگارش

آتنا اورنگی

خرداد ۱۴۰۵

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

چکیده

در این پروژه، یک الگوریتم مسیریابی بهبودیافته با نام $EBS-A^*$ (ترکیبی از بهینه‌سازی فاصله توسعه، جستجوی دوطرفه و هموارسازی مسیر) ارائه شده است. الگوریتم سنتی A^* علیرغم کارایی مناسب در یافتن کوتاه‌ترین مسیر، دارای محدودیت‌هایی از جمله نزدیکی بیش از حد به موانع، زمان محاسباتی بالا در نقشه‌های بزرگ و تولید مسیرهای ناهموار با زوایای تند 90° درجه است. برای رفع این مشکلات، ابتدا مفهوم «فاصله توسعه» برای ایجاد حریم ایمن در اطراف موانع پیاده‌سازی شد. سپس با استفاده از جستجوی دوطرفه همزمان از مبدأ و مقصد، فضای جستجو کاهش یافته و تعداد گره‌های بررسی‌شده به طور چشمگیری کم شد. در نهایت، با به‌کارگیری منحنی‌های بزیه، مسیر نهایی هموار شده و زوایای تند به گردش‌های نرم‌تر 45° درجه تبدیل شدند. الگوریتم پیشنهادی در نقشه‌هایی با ابعاد 50×50 ، 100×100 و 150×150 و در دو حالت نقشه ثابت و نقشه با موانع تصادفی شبیه‌سازی شد. نتایج نشان داد که الگوریتم $EBS-A^*$ نسبت به A^* سنتی تا 7.4 برابر سریع‌تر اجرا می‌شود، تعداد گره‌های بحرانی مجاور موانع را از 37 گره به 3 گره کاهش می‌دهد و مسیری کاملاً هموار بدون هیچ زاویه تند 90° درجه تولید می‌کند.

واژه‌های کلیدی:

الگوریتم مسیریابی A^* ، فاصله توسعه، جستجوی دوطرفه، هموارسازی مسیر

فهرست مطالب

صفحه

عنوان

۱		۱ مقدمه
۲		۱-۱ ساختار کلی پایان نامه
۴		۲ مرور ادبیات
۷		۳ بهبود الگوریتم A^*
۸		۱-۳ نظریه پایه الگوریتم سنتی A^*
۱۱		۲-۳ روش فاصله‌ی توسعه
۱۲		۳-۳ روش بهینه‌سازی جستجوی دوطرفه
۱۳		۴-۳ روش بهینه‌سازی هموارسازی مسیر
۱۶		۵-۳ الگوریتم $EBS-A^*$
۱۶		۱-۵-۳ روند اجرای الگوریتم
۱۷		۲-۵-۳ تحلیل پیچیدگی زمانی الگوریتم $EBS-A^*$
۱۸		۴ شبیه‌سازی الگوریتم‌ها
۲۶		۵ نتیجه‌گیری
۲۸		کتاب‌نامه

شکل	فهرست تصاویر	صفحه
۱-۳	فلوچارت الگوریتم	۱۰
۲-۳	نمودار شماتیک فاصله توسعه	۱۱
۳-۳	نمودارهای شماتیک جستجوی دوطرفه	۱۳
۴-۳	استراتژی بهینه‌سازی هموارسازی برای گردش‌های متوالی قائم	۱۵
۵-۳	مراحل اجرای الگوریتم EBS-A*	۱۷
۱-۴	نتایج شبیه‌سازی چهار الگوریتم روی یک نقشه ۵۰ در ۵۰	۱۹
۲-۴	نتایج شبیه‌سازی چهار الگوریتم روی یک نقشه ۱۰۰ در ۱۰۰	۲۲
۳-۴	نتایج شبیه‌سازی چهار الگوریتم روی یک نقشه ۱۵۰ در ۱۵۰	۲۴

فهرست جداول

صفحه

جدول

۲۰	۱-۴	نتایج شبیه‌سازی چهار الگوریتم روی نقشه ۵۰×۵۰
۲۱	۲-۴	میانگین نتایج شبیه‌سازی چهار الگوریتم روی نقشه تصادفی ۵۰×۵۰
۲۳	۳-۴	نتایج شبیه‌سازی چهار الگوریتم روی نقشه ۱۰۰×۱۰۰
۲۳	۴-۴	میانگین نتایج شبیه‌سازی چهار الگوریتم روی نقشه تصادفی ۱۰۰×۱۰۰
۲۵	۵-۴	نتایج شبیه‌سازی چهار الگوریتم روی نقشه ۱۵۰×۱۵۰
۲۵	۶-۴	میانگین نتایج شبیه‌سازی چهار الگوریتم روی نقشه تصادفی ۱۵۰×۱۵۰

فصل اول

مقدمه

در رباتیک، یکی از مهم‌ترین چالش‌ها در مسیریابی ربات‌ها، برنامه‌ریزی مسیر است. هدف از این فرآیند این است که ربات‌ها بتوانند از نقطه شروع به هدف خود برسند بدون اینکه با موانع موجود در محیط برخورد کنند. الگوریتم‌های مختلفی برای حل این مشکل توسعه یافته‌اند که یکی از معروف‌ترین آن‌ها الگوریتم A^* است. الگوریتم A^* به دلیل توانایی در یافتن مسیر بهینه در فضای جستجو، به‌طور گسترده‌ای مورد استفاده قرار می‌گیرد. اما با وجود مزایای فراوان، این الگوریتم با چالش‌هایی مواجه است، از جمله زمان محاسبه طولانی در محیط‌های پیچیده و احتمال مسیره‌های ناپایدار در نزدیکی موانع. برای مقابله با این مشکلات، الگوریتم $EBS-A^*$ معرفی شده است که به‌طور خاص برای بهبود الگوریتم A^* طراحی شده است. این الگوریتم از سه استراتژی کلیدی بهره می‌برد که به بهبود عملکرد و سرعت جستجو کمک می‌کند: فاصله گسترش، جستجوی دوطرفه و صاف‌سازی مسیر.

- فاصله گسترش^۱ باعث ایجاد ناحیه‌ای امن اطراف موانع می‌شود که از برخورد ربات با موانع جلوگیری می‌کند.

- جستجوی دوطرفه^۲ به جستجو از هر دو سمت شروع و هدف به طور همزمان پرداخته و سرعت جستجو را افزایش می‌دهد.

- صاف‌سازی مسیر^۳ با حذف پیچ‌های تند، مسیر بهینه‌تری برای حرکت ربات فراهم می‌کند. این سه استراتژی باعث بهبود چشمگیر الگوریتم $EBS-A^*$ در برنامه‌ریزی مسیر ربات‌ها شده و عملکرد آن را در محیط‌های پیچیده بهبود می‌بخشد.

۱-۱ ساختار کلی پایان‌نامه

در این بخش، ساختار کلی پروژه و ارتباط میان فصل‌ها و بخش‌های اصلی آن ارائه می‌شود. همان‌گونه که در شکل مشاهده می‌شود، این پایان‌نامه از چند بخش اصلی تشکیل شده است. ابتدا در فصل اول، مقدمه، بیان مسئله و اهداف پژوهش مطرح می‌شود. سپس در فصل دوم، مبانی نظری و پژوهش‌های مرتبط مورد بررسی قرار می‌گیرند. در فصل سوم، روش‌ها و الگوریتم‌های مورد استفاده، به‌ویژه الگوریتم A^* و نسخه‌های بهبودیافته‌ی آن، تشریح می‌شوند. در ادامه، فصل چهارم به شبیه‌سازی، ارزیابی و مقایسه‌ی نتایج اختصاص دارد و در نهایت، در فصل پنجم، جمع‌بندی نهایی و پیشنهادهایی برای پژوهش‌های آینده

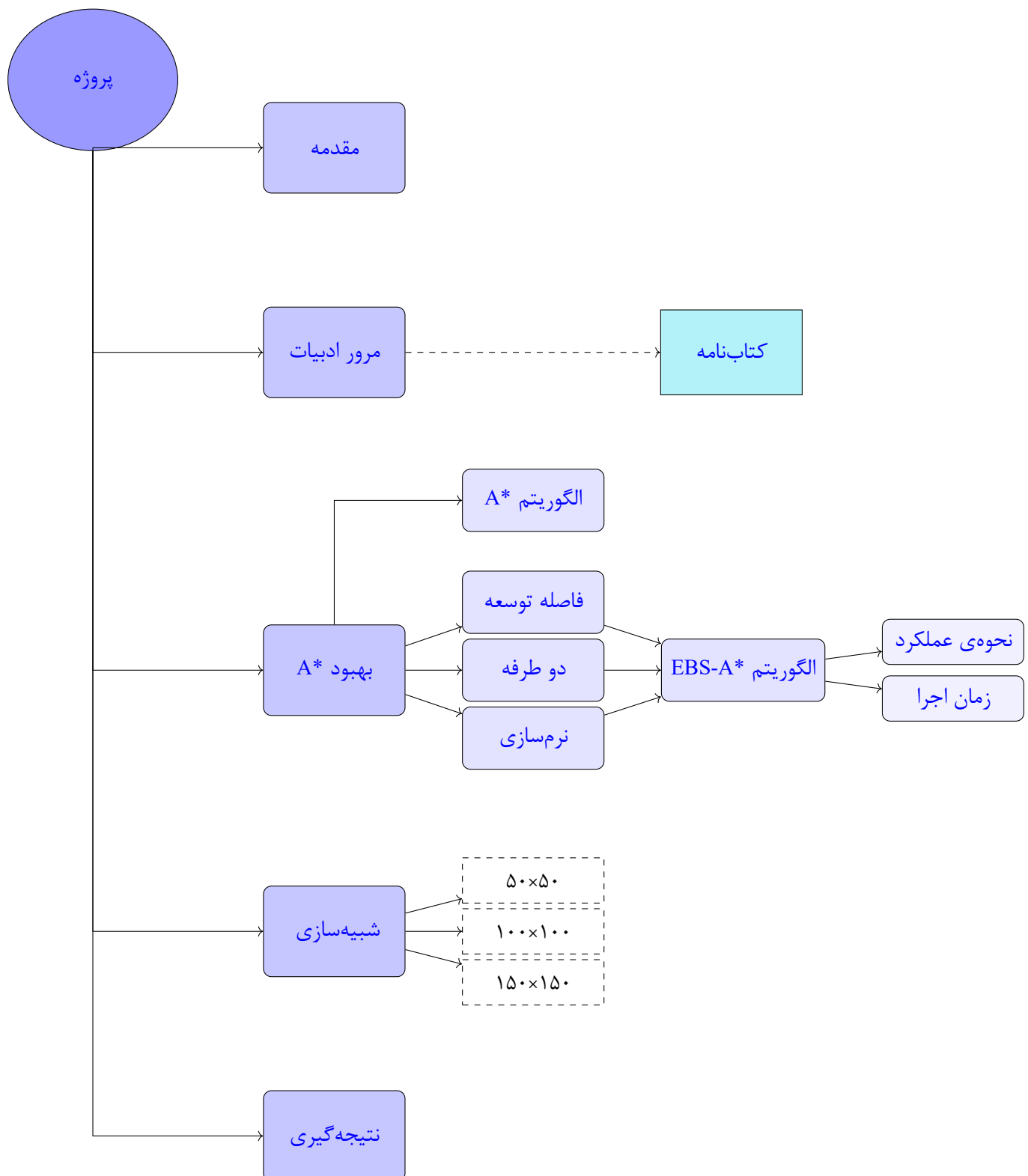
¹ Expansion distance

² Bidirectional Search Optimization

³ Smoothing Optimization

ارائه می‌شود.

ساختار کلی پروژه در شکل زیر نشان داده شده است.



فصل دوم

مرور ادبیات

الگوریتم‌های مسیریابی بر اساس میزان شناخت از محیط به دو دسته سراسری^۱ (محیط ایستا/شناخته‌شده) و محلی^۲ (محیط پویا/ناشناخته) تقسیم می‌شوند. همچنین می‌توان آن‌ها را به الگوریتم‌های جستجوی کلاسیک و الگوریتم‌های اکتشافی دسته‌بندی کرد. الگوریتم A^* یکی از پایه‌ای‌ترین روش‌ها برای ارزیابی کوتاه‌ترین مسیر در محیط‌های ایستا است [۴، ۱]. در حالی که الگوریتم‌های پویا برای چنین محیط‌هایی کارایی ندارند.

الگوریتم دیجسترا^۳ اگرچه کوتاه‌ترین مسیر را می‌یابد، اما به دلیل جستجوی کورکورانه و پیمایش کامل نقشه، محاسبات سنگین و کارایی پایینی دارد [۴]. الگوریتم A و نسخه‌های مختلف آن به عنوان الگوریتم‌های پیشرفته برای غلبه بر این محاسبات سنگین در محیط‌های ایستا معرفی شدند [۵]. با این حال، A^* نیز به دلیل نیاز به بررسی گره‌های متعدد، زمان محاسبه بالایی دارد که با بزرگ‌شدن مقیاس نقشه، کارایی آن کاهش می‌یابد [۱۰]. علاوه بر این، استفاده از نقشه‌های شبکه‌ای^۴ در این الگوریتم منجر به تولید مسیرهای ناصاف با چرخش‌های قائم‌الزاویه متعدد می‌شود که قابلیت اطمینان آن را پایین می‌آورد. این الگوریتم برای محاسبه هزینه مسیر از توابع اکتشافی نظیر فاصله منتهی [۸]، فاصله اقلیدسی و فاصله قطری [۱۱] بهره می‌برد.

محققان روش‌های مختلفی برای بهبود کارایی A^* ارائه داده‌اند؛ از جمله: وزن‌دهی به توابع ارزیابی و علامت‌گذاری گره‌های مجاور موانع به عنوان «نقاط نامطمئن» [۷]، بهینه‌سازی نحوه ذخیره‌سازی گره‌ها در آرایه‌ها برای جستجوی سریع‌تر [۶] و محاسبه زود هنگام توابع اکتشافی برایم‌های زمان اجرا [۲]؛ و هیچ‌کدام به بهبود استحکام^۵ A^* الگوریتم نپرداخته‌اند.

برای جلوگیری از برخورد، مفهوم «فاصله ایمن» با ایجاد یک مرز دایره‌ای حول وسایل نقلیه سطحی پیشنهاد شده است. هر چند این روش موانع نقشه را به خوبی لحاظ نمی‌کند. برای رفع مشکل ناصافی مسیرها در A^* ، روش‌هایی مانند ادغام مسیریابی سراسری با ویژگی‌های محیط محلی و حذف گره‌های اضافی ارائه شده است [۱۲]. همچنین استفاده از تکنیک‌های هموارسازی مانند درون‌یابی اسپالین مکعبی^۶ [۹]، برای کاهش چرخش‌های تند و خطر برخورد بسیار مؤثر بوده‌اند. علاوه بر این، نسخه‌های اصلاح‌شده‌ای مانند Basic Theta و JPS [۳] و همچنین A^* برای وسایل نقلیه خودران توسعه یافته‌اند

¹global²local³Dijkstra⁴rasterizes maps⁵robustness⁶Cubic Spline Interpolation

که امکان یافتن مسیر بهینه را فراهم می‌کنند.

روش‌های فعلی بهبودیافته از الگوریتم A^* ، تنها روی یکی از دو فاکتور «کارایی» یا «استحکام» تمرکز کرده‌اند. با توجه به کاربرد گسترده‌های متحرک در دنیای واقعی، جای خالی الگوریتمی جامع که همزمان دارای استحکام قوی و کارایی بالا باشد، احساس می‌شود؛ موضوعی که پتانسیل کاربردی و ارزش تجاری بالایی در بخش صنعت دارد.

فصل سوم

بهبود الگوریتم A^*

۱-۳ نظریه پایه الگوریتم سنتی A^*

الگوریتم A^* یک روش جستجوی هیوریستیک^۱ و بهترین-اول^۲ برای یافتن کم‌هزینه‌ترین مسیر است که نسبت به جستجوهای کور مزیت دارد. نکات کلیدی آن عبارتند از:

نحوه عملکرد: این الگوریتم با استفاده از یک جدول باز و یک جدول بسته، گره‌ها را بر اساس تابع ارزیابی زیر بررسی و انتخاب می‌کند.

$$f(n) = g(n) + h(n) \quad (۱-۳)$$

در رابطه (۱-۳) جمله‌ی $g(n)$ نمایانگر هزینه طی شده از مبدأ تا گره فعلی، $h(n)$ تخمین هزینه از گره فعلی تا مقصد و $f(n)$ برابر با هزینه کل ارزیابی شده برای گره است.

مزیت اصلی: این الگوریتم همواره گره‌ای با کمترین مقدار $f(n)$ را انتخاب می‌کند و انعطاف‌پذیری بالایی در استفاده از انواع فاصله‌های هیوریستیک (مانند اقلیدسی یا منهتن) دارد.

شبه‌کد این الگوریتم را در صفحه‌ی بعد مشاهده می‌کنید.

^۱heuristic^۲best-first

Algorithm 1 A* Pathfinding Algorithm

Input: Start node *start*, goal node *goal*, heuristic function $h(n)$, graph with edge costs $d(u, v)$

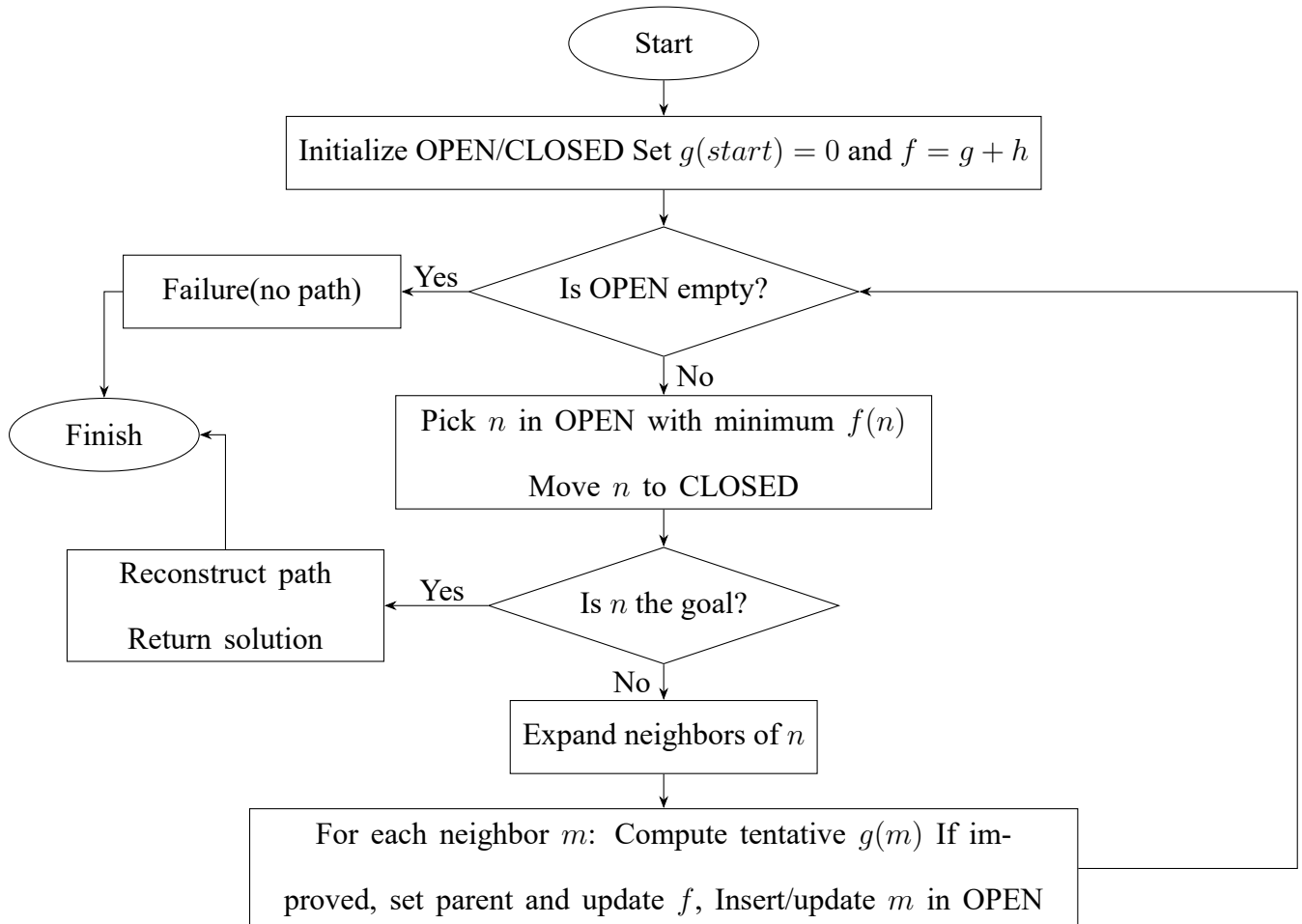
Output: Shortest path from *start* to *goal* as a list of nodes, or *failure* if no path exists

```

1: function RECONSTRUCT_PATH(came_from, current)
2:   total_path  $\leftarrow \{current\}$ 
3:   while current in came_from.keys do
4:     current  $\leftarrow$  came_from[current]
5:     total_path.prepend(current)
6:   end while
7:   return total_path
8: end function
9: function A_STAR(start, goal, h)
10:  open_set  $\leftarrow \{start\}$ 
11:  came_from  $\leftarrow$  empty map
12:  g_score  $\leftarrow$  map with default value of  $\infty$ 
13:  g_score[start]  $\leftarrow 0$ 
14:  f_score  $\leftarrow$  map with default value of  $\infty$ 
15:  f_score[start]  $\leftarrow h(start)$ 
16:  while open_set  $\neq \emptyset$  do
17:    current  $\leftarrow$  node in open_set with lowest f_score
18:    if current = goal then
19:      return RECONSTRUCT_PATH(came_from, current)
20:    end if
21:    open_set.remove(current)
22:    for each neighbor n of current do
23:      tentative_g  $\leftarrow g\_score[current] + d(current, n)$ 
24:      if tentative_g < g_score[n] then
25:        came_from[n]  $\leftarrow$  current
26:        g_score[n]  $\leftarrow$  tentative_g
27:        f_score[n]  $\leftarrow$  tentative_g + h(n)
28:        if n  $\notin$  open_set then
29:          open_set.add(n)
30:        end if
31:      end if
32:    end for
33:  end while
34:  return failure
35: end function

```

با توجه به الگوریتم ۱، اجرای $path \leftarrow A^*(start, goal, h)$ باعث می‌شود کوتاه‌ترین مسیر از گره شروع به گره هدف با استفاده از تابع هیوریستیک h پیدا شود. خروجی این الگوریتم، مسیر به صورت لیستی از گره‌ها است که در صورت نبود مسیر، مقدار $failure$ بازگردانده می‌شود. برای درک بهتر مراحل این الگوریتم، فلوچارت آن در شکل ۱-۳ ارائه شده است.



شکل ۱-۳: فلوچارت الگوریتم

همان‌طور که در شکل ۱-۳ مشاهده می‌شود، الگوریتم A^* با قرار دادن گره شروع در لیست باز آغاز شده و تا خالی شدن لیست باز یا رسیدن به گره هدف ادامه می‌یابد. در هر مرحله، گره با کمترین مقدار $f(n)$ از لیست باز انتخاب شده و همسایگان آن بررسی می‌شوند. این روند تا رسیدن به جواب یا شکست ادامه پیدا می‌کند.

در ادامه، روش‌های بهبود الگوریتم A^* ارائه می‌شود.

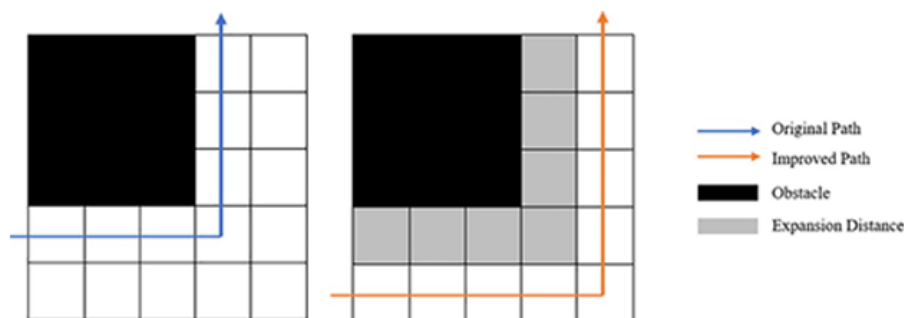
۲-۳ روش فاصله‌ی توسعه

الگوریتم سنتی A^* ممکن است مسیرهایی بسیار نزدیک به موانع تولید کند که خطر برخورد ربات را افزایش می‌دهد. برای رفع این مشکل، از مفهوم «فاصله توسعه» به عنوان یک ناحیه ایمن^۱ در اطراف موانع استفاده می‌شود. در این روش، گره‌هایی که در مجاورت موانع قرار دارند از فرآیند ارزیابی و جستجو حذف می‌شوند. این کار نه تنها مسیرهای ایمن‌تری ایجاد می‌کند، بلکه با کاهش تعداد گره‌های قابل بررسی، زمان محاسبات را کم کرده و کارایی الگوریتم را بهبود می‌بخشد.

اندازه این فاصله در حالت پایه برابر با شعاع مدل ربات r است، اما در محیط‌های پویا این فاصله متغیر بوده و به سرعت ربات V_i بستگی دارد. قانون تعیین فاصله توسعه بر اساس سرعت آستانه V_p و سرعت حداکثر V_{max} طبق فرمول زیر محاسبه می‌شود:

$$E(V_i) = \begin{cases} r & V_i \leq V_r \\ \frac{V_i}{V_r} r & V_r \leq V_i \leq V_{max} \end{cases} \quad (2-3)$$

همان‌طور که در رابطه (۲-۳) مشاهده می‌شود، فاصله توسعه برای سرعت‌های کمتر از آستانه V_r برابر با شعاع ربات r در نظر گرفته می‌شود. با افزایش سرعت ربات از V_r تا حداکثر سرعت V_{max} ، این فاصله به صورت خطی افزایش می‌یابد که باعث می‌شود ربات در سرعت‌های بالا، ناحیه ایمن بزرگتری را در اطراف موانع رعایت کند. این مکانیزم تطبیقی، تعادل مناسبی بین ایمنی و کارایی مسیر ایجاد می‌نماید.



شکل ۲-۳: نمودار شماتیک فاصله توسعه

¹buffer

۳-۳ روش بهینه‌سازی جستجوی دوطرفه

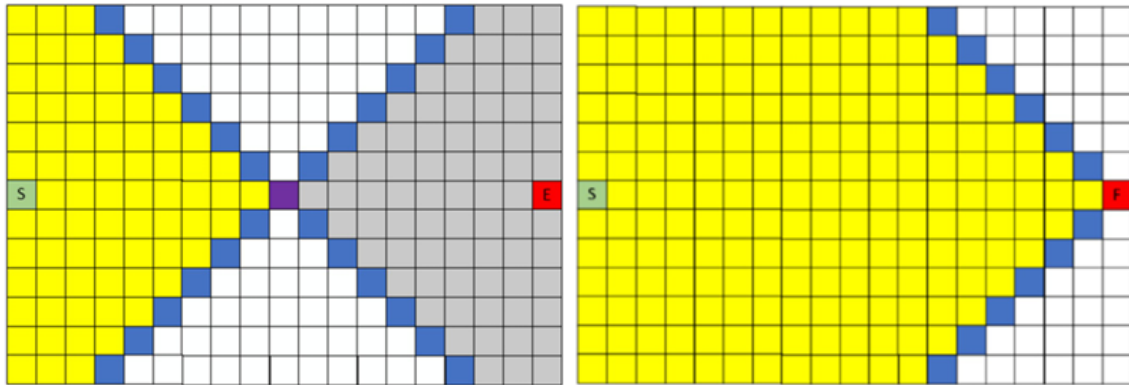
الگوریتم A^* از شناخته‌شده‌ترین الگوریتم‌های برنامه‌ریزی مسیر است که برای یافتن کوتاه‌ترین مسیر بین نقطه شروع و هدف در گراف‌ها استفاده می‌شود. یکی از مشکلات اصلی این الگوریتم، پیچیدگی محاسباتی زیاد در گراف‌های بزرگ و پیچیده است که باعث افزایش زمان جستجو می‌شود.

برای رفع این مشکل، روش بهینه‌سازی جستجوی دوطرفه معرفی شده است. در این روش، جستجو همزمان از هر دو سمت گره شروع و گره هدف انجام می‌شود و مسیرهای جستجو به سمت یکدیگر پیش می‌روند تا در میانه راه به هم برسند و مسیر کامل ساخته شود. این کار باعث کاهش فضای جستجو و کاهش تعداد گره‌های مورد بررسی می‌شود و به طبع آن سرعت اجرای الگوریتم افزایش می‌یابد.

جستجوی دوطرفه بر اساس ارزیابی تابع هزینه کلی که در رابطه (۳-۱) نشان داده شده است، انجام می‌شود. هر گره به عنوان گره بعدی که کمترین مقدار $f(n)$ را دارد انتخاب می‌شود، ولی این انتخاب به صورت همزمان از دو طرف انجام می‌شود.

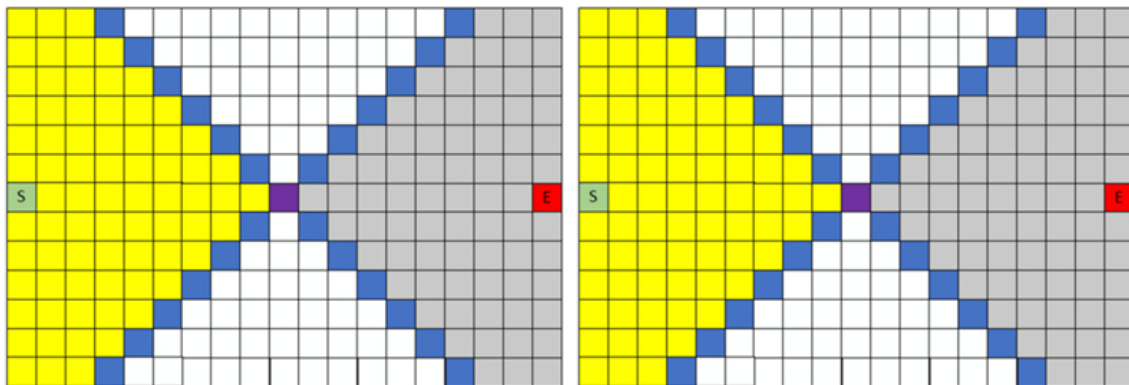
همچنین با استفاده از این روش، فضای جستجو کاهش یافته و تعداد فراخوانی‌های تابع به طور چشمگیری کمتر خواهد شد، خصوصاً در گراف‌های بسیار بزرگ که جستجوی یک‌طرفه زمان زیادی می‌برد.

شکل ۳-۳ نمونه‌هایی از بهینه‌سازی جستجوی دوطرفه را در گراف‌های مختلف نشان می‌دهد که چگونه جستجو از دو سمت باعث کاهش فضای جستجو و افزایش سرعت پیدا کردن مسیر می‌شود.



(ب) جستجوی دوطرفه بدون مانع

(آ) نمودار شماتیک جستجوی یک طرفه



(د) جستجوی دوطرفه با موانع متعدد

(ج) جستجوی دوطرفه با یک مانع

شکل ۳-۳: نمودارهای شماتیک جستجوی دوطرفه

همان طور که در شکل ۳-۳ مشاهده می شود، جستجوی دوطرفه می تواند در شرایط مختلف عملکرد متفاوتی داشته باشد. در زیر شکل ۳-۳، جستجوی یک طرفه نشان داده شده است که در آن مسیر فقط از مبدأ به سمت هدف گسترش می یابد. در مقابل، زیر شکل های ب و ج و د از شکل ۳-۳ به ترتیب جستجوی دوطرفه را در حالت بدون مانع، با یک مانع و با موانع متعدد نمایش می دهند. این مقایسه نشان می دهد که جستجوی دوطرفه با کاهش فضای جستجو، به ویژه در حضور موانع، کارایی بالاتری نسبت به حالت یک طرفه دارد.

۴-۳ روش بهینه سازی هموارسازی مسیر

مسیر تولید شده توسط الگوریتم A* شامل قطعات خطی به هم پیوسته است که سه ایراد عمده دارد:

- مسیر نهایی ناهموار است و زاویه های تند (عموماً ۹۰ درجه) در آن وجود دارد.

- مسیر تولید شده پیوسته نیست.

- وجود زاویه‌های تند منجر به کاهش سرعت حرکت ربات در آن نقاط می‌شود که باعث کاهش کارایی کل مسیر می‌گردد.

برای رفع این مشکلات، از روش هموارسازی مسیر استفاده می‌شود که در آن مسیر اصلی با یک منحنی نرم جایگزین می‌شود. یکی از بهترین روش‌ها برای این کار، استفاده از منحنی بزیه^۱ است. منحنی بزیه دارای ویژگی‌های ریاضی مهمی است که از جمله مهمترین آنها خاصیت بازگشتی ضرایب برنشتاین^۲ می‌باشد. بر اساس این ویژگی، ضرایب منحنی بزیه از رابطه بازگشتی (۳-۳) پیروی می‌کنند که محاسبه نقاط روی منحنی را بدون نیاز به محاسبه مستقیم ضرایب دوجمله‌ای امکان‌پذیر می‌سازد.

$$\begin{aligned}
 B_{i,n}(t) &= (1-t)B_{i,n-1}(t) + tB_{i-1,n-1}(t) \\
 &= (1-t)[(1-t)B_{i,n-2}(t) + tB_{i-1,n-2}(t)] + t[(1-t)B_{i-1,n-2}(t) + tB_{i-2,n-2}(t)] \\
 &= (1-t)^2 B_{i,n-2}(t) + 2t(1-t)B_{i-1,n-2}(t) + t^2 B_{i-2,n-2}(t) \\
 &= \sum_{k=0}^m \binom{m}{k} t^k (1-t)^{m-k} B_{i-k,n-m}(t), \quad m \leq i \leq n
 \end{aligned} \tag{۳-۳}$$

بر اساس تعریف چندجمله‌ای‌های برنشتاین، منحنی بزیه از درجه n به صورت پارامتری و با ترکیب این چندجمله‌ای‌ها تعریف می‌شود. رابطه (۴-۳) نمایش کلی یک منحنی بزیه را بر حسب نقاط کنترل و چندجمله‌ای‌های پایه نشان می‌دهد.

$$P(t) = \sum_{i=1}^n p_i B_{i,n}(t), \quad t \in [0, 1] \tag{۴-۳}$$

در این رابطه، $P_i(x_i, y_i)^T$ بردار مختصات i -امین نقطه کنترل و $B_{i,n}(t)$ چندجمله‌ای پایه برنشتاین است که به صورت زیر تعریف می‌شود:

$$B_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}, \quad i = 0, 1, \dots, n \tag{۵-۳}$$

یکی از دلایل اصلی محبوبیت منحنی بزیه در مسائل برنامه‌ریزی مسیر، مشتق‌پذیری پیوسته آن

^۱Bezier Curve

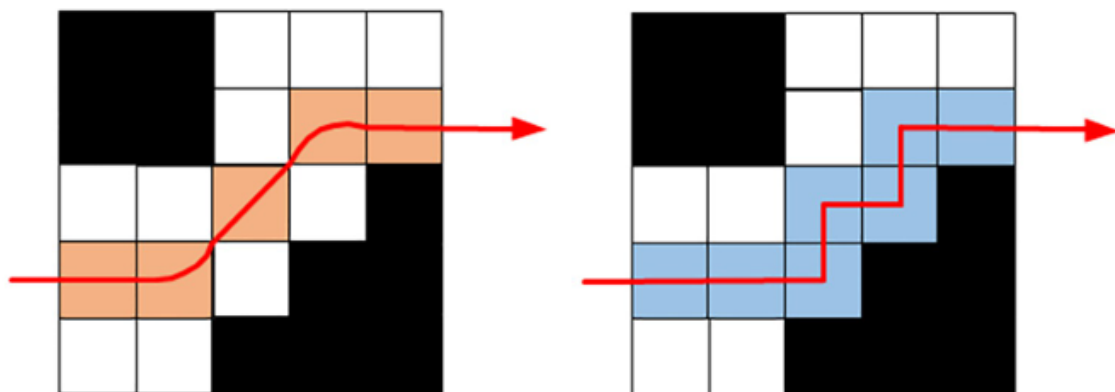
^۲Bernstein

است. مشتق مرتبه اول این منحنی بر حسب نقاط کنترل به صورت رابطه (۶-۳) قابل محاسبه می‌باشد که برای تعیین جهت حرکت ربات در طول مسیر کاربرد دارد.

$$\dot{P}(t) = \frac{dP(t)}{dt} = n \sum_{i=0}^{n-1} B_{i,n-1}(t)(P_{i+1} - P_i) \quad (6-3)$$

به این ترتیب، با استفاده از منحنی بزیه می‌توان یک مسیر هموار با انحنای پیوسته ایجاد کرد که برای حرکت ربات‌های خودران بسیار مناسب است.

استراتژی اصلی بهبود مسیر، تقسیم گردش‌های تند ۹۰ درجه به دو گردش مالیم‌تر ۴۵ درجه است تا مسیر نرم‌تر و پیوسته شود و سرعت حرکت ربات حفظ گردد. هنگامی که گردش‌های متوالی ۹۰ درجه وجود داشته باشد، می‌توان نقطه شکست را حذف کرد و دو گره مجاور آن را مستقیماً به یکدیگر متصل نمود تا چندین گردش قائمه به تعداد کمی گردش حاد ۴۵ درجه تبدیل شوند، همانطور که در شکل ۴-۳ نشان داده شده است.



(ب) گردش ۴۵ درجه پس از بهینه‌سازی

(آ) گردش ۹۰ درجه

شکل ۴-۳: استراتژی بهینه‌سازی هموارسازی برای گردش‌های متوالی قائم

در زیرشکل آ یک گردش ۹۰ درجه مشاهده می‌شود که دارای زاویه تند و ناهموار است. پس از اعمال روش هموارسازی، این گردش به دو گردش ۴۵ درجه در زیرشکل ب تبدیل می‌شود که مسیر نرم‌تر و پیوسته‌تری را ارائه می‌دهد.

۵-۳ الگوریتم EBS-A*

۱-۵-۳ روند اجرای الگوریتم

شبه‌کد الگوریتم EBS-A* در زیر آورده شده است.

Algorithm 2 EBS-A* Algorithm

Input: Start node $Start_i$, goal node End_i , environment map Map , estimated cost $f(n)$

Output: Path $PATH$

```

1: Initialize  $Map$ ,  $PATH\_S$ ,  $PATH\_E$ 
2: Call  $ExpansionDistance(1)$ 
3: Create forward open list  $OPEN\_LIST\_1$  and forward close list  $CLOSE\_LIST\_1$ 
4: Create backward open list  $OPEN\_LIST\_2$  and backward close list  $CLOSE\_LIST\_2$ 
5: Add  $Start_i$  to  $OPEN\_LIST\_1$  and  $End_i$  to  $OPEN\_LIST\_2$ 
6: repeat
7:    $node\_s \leftarrow OPEN\_LIST\_1.removeNext()$ 
8:    $PATH\_S.add(node\_s)$ 
9:    $search(node\_s, OPEN\_LIST\_1, CLOSE\_LIST\_1)$ 
10:   $CLOSE\_LIST\_1.add(node\_s)$ 
11:   $node\_e \leftarrow OPEN\_LIST\_2.removeNext()$ 
12:   $PATH\_E.add(node\_e)$ 
13:   $search(node\_e, OPEN\_LIST\_2, CLOSE\_LIST\_2)$ 
14:   $CLOSE\_LIST\_2.add(node\_e)$ 
15: until  $node\_s = End_i$  or  $node\_e = Start_i$  or  $OPEN\_LIST\_1 = \emptyset$  or
       $OPEN\_LIST\_2 = \emptyset$  or  $GetNeighbor(node\_s) \cap GetNeighbor(node\_e) \neq \emptyset$ 
16:  $PATH \leftarrow PATH\_S + reverse(PATH\_E)$ 
17:  $smooth(PATH)$ 
18: return  $PATH$ 

```

فرآیند اجرایی الگوریتم EBS-A* شامل چهار مرحله است:

۱. انجام بهینه‌سازی فاصله توسعه روی الگوریتم A*.

۲. انجام جستجوی دوطرفه روی الگوریتم.

۳. تولید مسیر نامنعطف (ناهموار).

۴. انجام فرآیند هموارسازی برای تولید مسیر هموار.

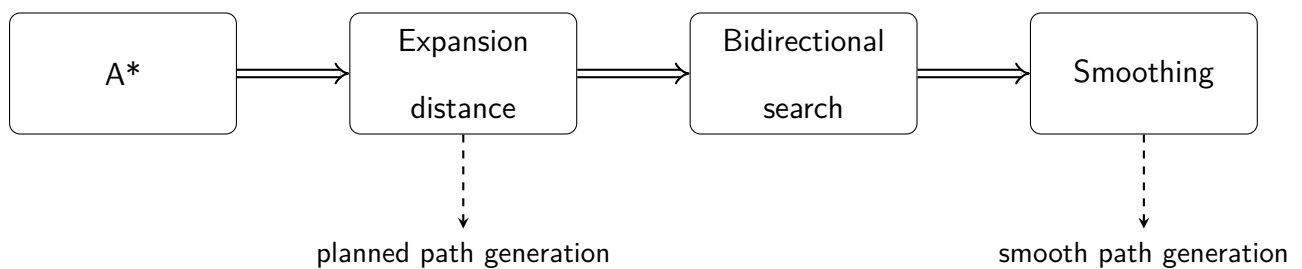
فرآیند اجرایی الگوریتم پیشنهادی در شکل ۵-۳ نشان داده شده است.

۳-۵-۲ تحلیل پیچیدگی زمانی الگوریتم $EBS-A^*$

در این بخش، با ترکیب جستجوی دوطرفه و بهینه‌سازی هموارسازی گردش‌های قائمه، شبه‌کد الگوریتم $EBS-A^*$ به صورت زیر ارائه می‌شود. این الگوریتم از یک حلقه دوگانه استفاده می‌کند. حلقه داخلی، گره‌های مجاور را در چهار جهت پیمایش کرده و گره با کمترین هزینه مشخص شده و به جدول باز اضافه می‌شود. حلقه خارجی، گره‌های جدول باز را تا پایان پیمایش صف، طی می‌کند. عوامل متعددی مانند مقیاس نقشه، موقعیت گره شروع و گره هدف، تأثیر آشکاری بر پیچیدگی زمانی الگوریتم‌های مختلف برنامه‌ریزی مسیر دارند. به طور کلی، پیچیدگی زمانی الگوریتم‌های مختلف به صورت زیر بیان می‌شود:

$$T \in [O(\min[m, n]), O(m \times n)] \quad (۷-۳)$$

در رابطه (۷-۳)، m و n به ترتیب طول و عرض نقشه هستند. از آنجا که جستجوی دوطرفه یک استراتژی جستجو در دو جهت است، هنگامی که مسیر شدنی باشد، مسیر تشکیل شده در هر جهت جستجو از کل مسیر کوچک‌تر است که این امر تعداد گره‌های اضافه‌شده به جدول باز و تعداد دفعات پیمایش حلقه را کاهش می‌دهد. بنابراین، پیچیدگی زمانی الگوریتم $EBS-A^*$ کمتر از یا مساوی نصف پیچیدگی زمانی الگوریتم سنتی A^* است.

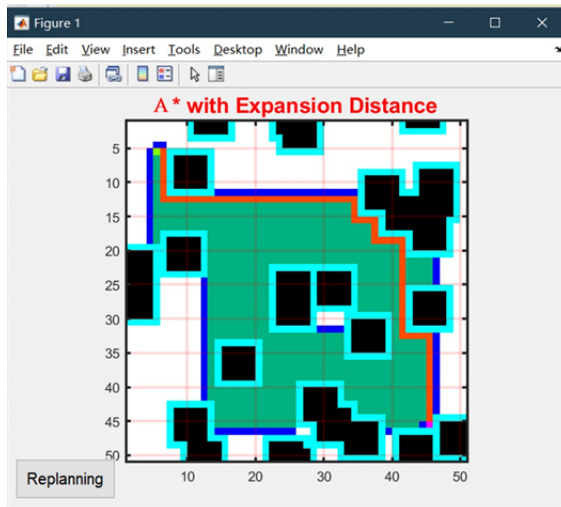


شکل ۳-۵: مراحل اجرای الگوریتم $EBS-A^*$

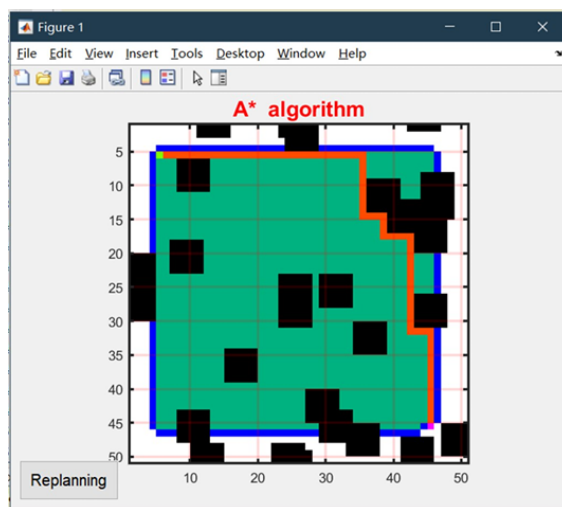
فصل چهارم

شبیه‌سازی الگوریتم‌ها

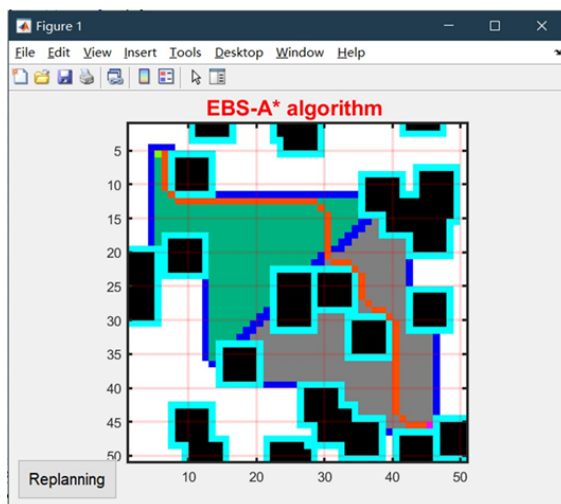
در این بخش، چهار الگوریتم شبیه‌سازی و تست شده‌اند: الگوریتم A^* سنتی، الگوریتم A^* با فاصله توسعه، الگوریتم A^* دوطرفه با فاصله توسعه، و الگوریتم $EBS-A^*$.



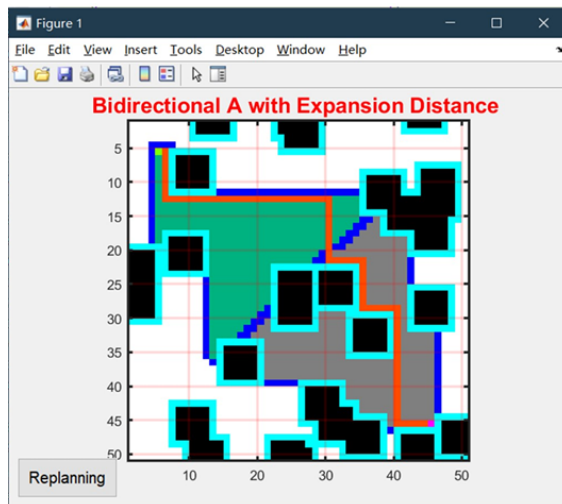
(ب) زمان سپری شده ۲۱,۳۳۶۴۸۷ ثانیه.
مسیر پیدا شد!
تعداد گره‌ها = ۷۹
گره گسترش یافته = ۷۸۷
 A^* با فاصله توسعه



(آ) زمان سپری شده ۳۶,۸۰۵۵۸۴ ثانیه.
مسیر پیدا شد!
تعداد گره‌ها = ۷۹
گره گسترش یافته = ۱۳۷۲
 A^* الگوریتم



(د) مسیر پیدا شد!
زمان سپری شده ۱۵,۴۱۱۷۵۸ ثانیه.
تعداد گره‌ها = ۶۶
گره گسترش یافته = ۶۸۲
 $EBS-A^*$ الگوریتم



(ج) مسیر پیدا شد!
زمان سپری شده ۱۴,۷۹۳۰۳۸ ثانیه.
تعداد گره‌ها = ۷۹
گره گسترش یافته = ۶۷۹
 A^* دوطرفه با فاصله توسعه

شکل ۴-۱: نتایج شبیه‌سازی چهار الگوریتم روی یک نقشه ۵۰ در ۵۰.

مقیاس نقشه در آزمایش شبیه‌سازی ۵۰×۵۰ است و اندازه هر مانع در نقشه ۵×۵ می‌باشد. موقعیت موانع به‌طور تصادفی بر اساس نقطه مرکز روی نقشه تولید می‌شود. چهار الگوریتم تست شده‌اند و

نتایج برنامه‌ریزی مسیر در شکل ۴-۱ نشان داده شده است. بررسی عملکرد چهار الگوریتم مسیریابی در نقشه‌های با ابعاد 50×50 نشان‌دهنده برتری قابل توجه الگوریتم EBS-A* در شاخص‌های کلیدی است:

- سرعت و کارایی: سرعت اجرای الگوریتم EBS-A* نسبت به A* سنتی حدود ۳۲۸٪ بهبود یافته و زمان اجرای آن تنها معادل ۲۶,۴۸٪ زمان الگوریتم A* سنتی است.
- همواری مسیر: EBS-A* با حذف کامل چرخش‌های زاویه قائمه (تعداد صفر در برابر ۷ چرخش در A*) و کاهش بیشترین زاویه چرخش از ۹۰ به ۴۵، مسیر بسیار نرم‌تر و پایدارتری ایجاد می‌کند.
- طول مسیر و ایمنی: این الگوریتم به کمترین تعداد گره برای برنامه‌ریزی مسیر نیاز دارد و تعداد گره‌های بحرانی (مجاور موانع) را از ۳۷ گره در A* سنتی به تنها ۳ گره کاهش داده است.
- گره‌های توسعه: تعداد گره‌های توسعه در این الگوریتم به ۶۸۲ مورد کاهش یافته که نشان‌دهنده کارایی بالای جستجو است.
- این نتایج هم در یک نقشه ثابت (جدول ۴-۱) و هم به صورت میانگین در نقشه‌های تصادفی (جدول ۴-۲) تأیید شده‌اند.

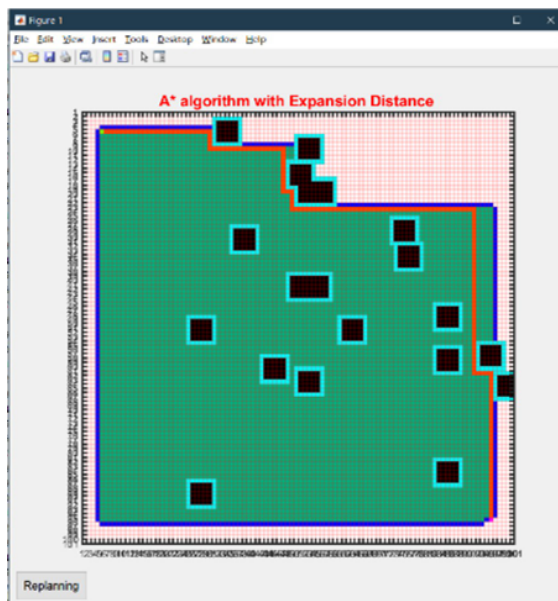
جدول ۴-۱: نتایج شبیه‌سازی چهار الگوریتم روی نقشه ۵۰×۵۰

شاخص‌ها	الگوریتم A*	A* با فاصله توسعه	A* دوطرفه با فاصله توسعه	الگوریتم EBS-A*
زمان اجرا (ثانیه)	۳۶,۸۰۶ (۱۰۰٪)	۲۱,۷۸۲ (۵۹,۱۸٪)	۹,۶۳۷ (۲۶,۱۸٪)	۹,۷۴۷ (۲۶,۴۸٪)
تعداد گره‌ها	۷۹	۷۹	۷۹	۶۶
تعداد گردش‌های ۹۰ درجه	۷	۹	۸	۰
حداکثر زاویه چرخش	۹۰°	۹۰°	۹۰°	۴۵°
تعداد گره‌های گسترش‌یافته	۱,۳۷۲	۷۸۷	۶۷۹	۶۸۲
تعداد گره‌های بحرانی	۳۷	۰	۰	۳

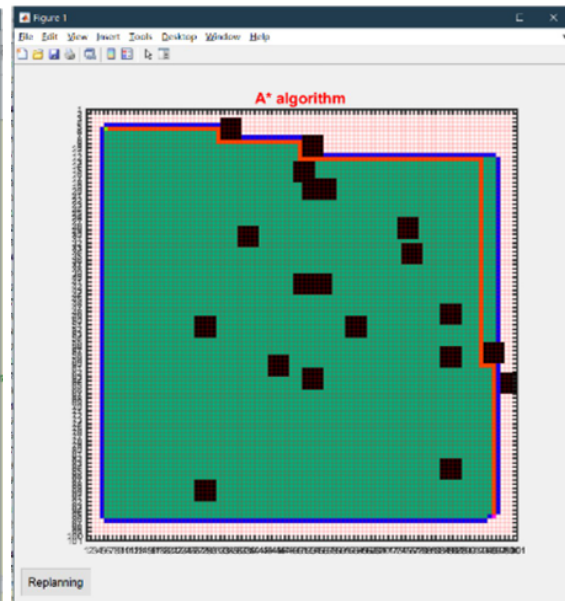
جدول ۴-۲: میانگین نتایج شبیه‌سازی چهار الگوریتم روی نقشه تصادفی ۵۰×۵۰

شاخص‌ها	الگوریتم A*	A* با فاصله توسعه	A* دوطرفه با فاصله توسعه	الگوریتم EBS-A*
زمان اجرا (ثانیه)	۳۳,۵۵۹(۱۰۰٪)	۲۱,۳۸۶(۶۳,۷۳٪)	۵,۶۳۹(۱۶,۸۰٪)	۷,۸۳۵(۲۳,۳۵٪)
تعداد گره‌ها	۷۹,۴	۷۹,۴	۷۹,۴	۶۷
تعداد گردش‌های ۹۰ درجه	۶,۴	۶,۲	۷	۰
حداکثر زاویه چرخش	۹۰°	۹۰°	۹۰°	۴۵°
تعداد گره‌های گسترش‌یافته	۱,۰۰۸۵	۷۴۴,۸	۵۵۳	۶۹۳,۸
تعداد گره‌های بحرانی	۳۶,۲	۰	۰	۳

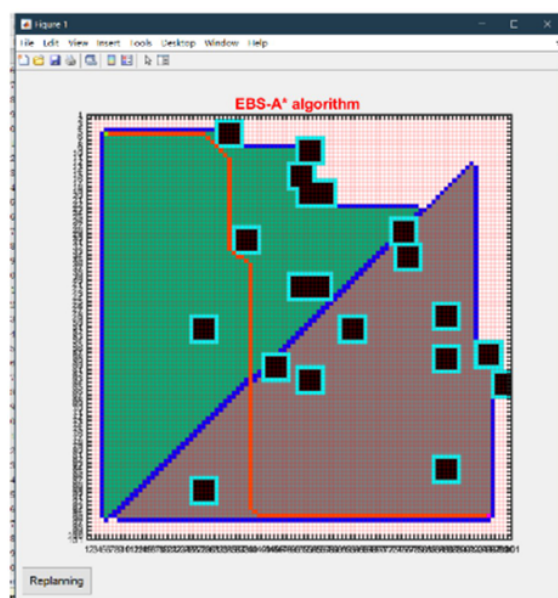
نتایج برنامه‌ریزی مسیر نقشه 100×100 در شکل ۴-۲ نشان داده شده است.



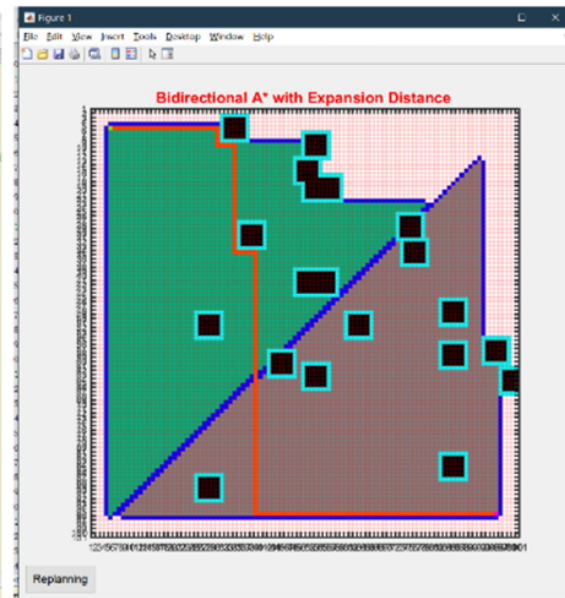
(ب) زمان سپری شده ۲۱,۳۳۶۴۸۷ ثانیه.
مسیر پیدا شد!
تعداد گره‌ها = ۷۹
گره گسترش یافته = ۷۸۷
با A* فاصله توسعه



(آ) زمان سپری شده ۳۶,۸۰۵۵۸۴ ثانیه.
مسیر پیدا شد!
تعداد گره‌ها = ۷۹
گره گسترش یافته = ۱۳۷۲
الگوریتم A*



(د) مسیر پیدا شد!
زمان سپری شده ۱۵,۴۱۱۷۵۸ ثانیه.
تعداد گره‌ها = ۶۶
گره گسترش یافته = ۶۸۲
الگوریتم EBS-A*



(ج) مسیر پیدا شد!
زمان سپری شده ۱۴,۷۹۳۰۳۸ ثانیه.
تعداد گره‌ها = ۷۹
گره گسترش یافته = ۶۷۹
A* دوطرفه با فاصله توسعه

شکل ۴-۲: نتایج شبیه‌سازی چهار الگوریتم روی یک نقشه 100×100 در ۱۰۰.

نتایج آماری برای نقشه‌ی واحد در جدول ۳-۴ و برای نقشه‌های تصادفی در جدول ۴-۴ آمده است.

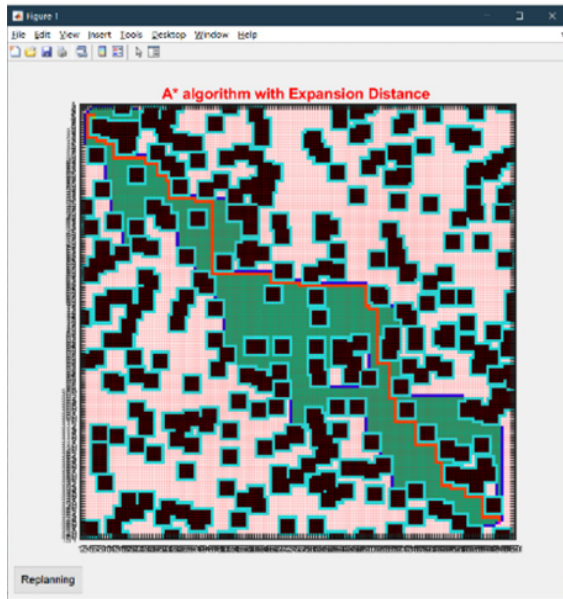
جدول ۳-۴: نتایج شبیه‌سازی چهار الگوریتم روی نقشه 100×100

شاخص‌ها	الگوریتم A*	A* با فاصله توسعه	A* دوطرفه با فاصله توسعه	الگوریتم EBS-A*
زمان اجرا (ثانیه)	۳۰۲,۴۶۷	۲۷۱,۱۹۰	۱۳۴,۷۱۹	۱۳۸,۸۱۳
تعداد گره‌ها	۱۷۹	۱۷۹	۱۷۹	۱۶۷
تعداد گردش‌های ۹۰ درجه	۷	۹	۶	۰
حداکثر زاویه چرخش	۹۰°	۹۰°	۹۰°	۴۵°
تعداد گره‌های گسترش‌یافته	۷,۵۰۹	۶,۷۲۲	۶,۵۳۱	۶,۵۳۳
تعداد گره‌های بحرانی	۳۱	۰	۰	۲

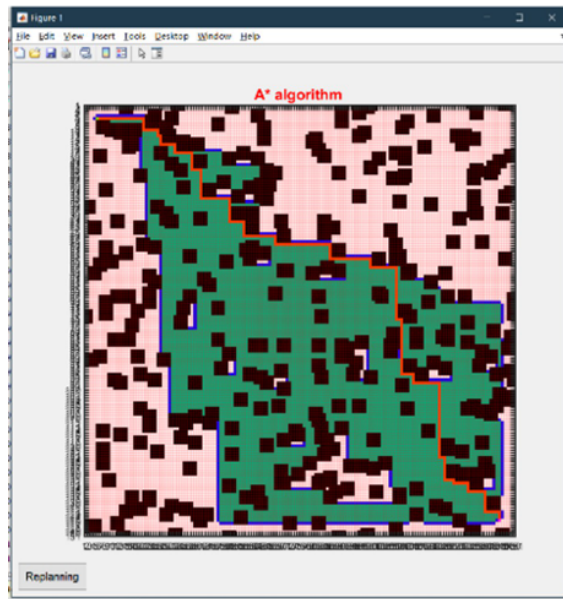
جدول ۴-۴: میانگین نتایج شبیه‌سازی چهار الگوریتم روی نقشه تصادفی 100×100

شاخص‌ها	الگوریتم A*	A* با فاصله توسعه	A* دوطرفه با فاصله توسعه	الگوریتم EBS-A*
زمان اجرا (ثانیه)	۲۶۹,۳۳۵	۲۶۸,۳۳۱	۱۲۸,۷۷۵	۱۲۵,۶۷۷
تعداد گره‌ها	۱۷۹	۱۷۹	۱۷۹	۱۷۰,۸
تعداد گردش‌های ۹۰ درجه	۶	۵,۷	۶	۰
حداکثر زاویه چرخش	۹۰°	۹۰°	۹۰°	۴۵°
تعداد گره‌های گسترش‌یافته	۷,۷۰۹,۴	۷,۳۰۲,۸	۶,۸۱۷,۴	۶,۵۹۹
تعداد گره‌های بحرانی	۱۰	۰	۰	۲

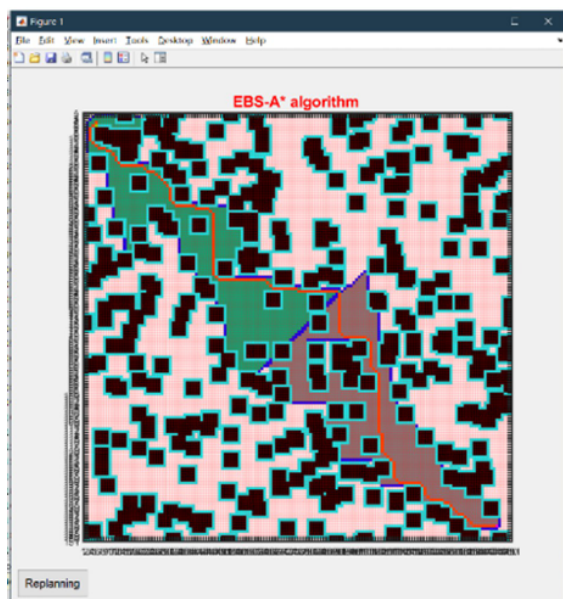
نتایج برنامه‌ریزی مسیر نقشه 150×150 در شکل زیر نشان داده شده است.



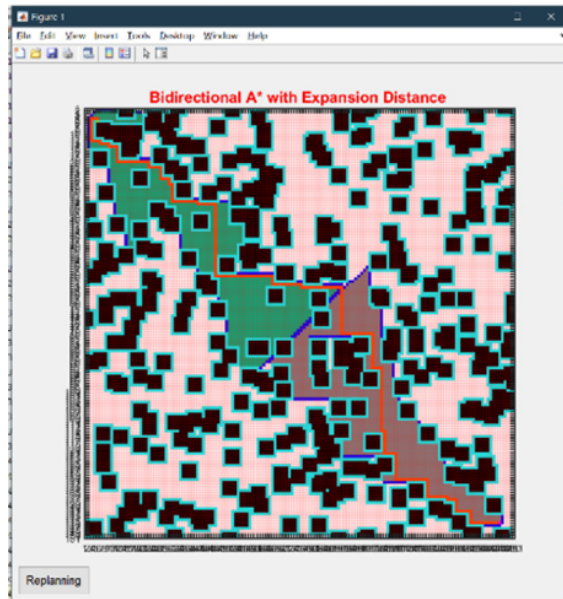
(ب) زمان سپری شده ۲۱,۳۳۶۴۸۷ ثانیه.
مسیر پیدا شد!
تعداد گره‌ها = ۷۹
گره گسترش‌یافته = ۷۸۷
A* با فاصله توسعه



(آ) زمان سپری شده ۳۶,۸۰۵۵۸۴ ثانیه.
مسیر پیدا شد!
تعداد گره‌ها = ۷۹
گره گسترش‌یافته = ۱۳۷۲
الگوریتم A*



(د) مسیر پیدا شد!
زمان سپری شده ۱۵,۴۱۱۷۵۸ ثانیه.
تعداد گره‌ها = ۶۶
گره گسترش‌یافته = ۶۸۲
الگوریتم EBS-A*



(ج) مسیر پیدا شد!
زمان سپری شده ۱۴,۷۹۳۰۳۸ ثانیه.
تعداد گره‌ها = ۷۹
گره گسترش‌یافته = ۶۷۹
A* دوطرفه با فاصله توسعه

شکل ۳-۴: نتایج شبیه‌سازی چهار الگوریتم روی یک نقشه ۱۵۰ در ۱۵۰.

نتایج آماری برای نقشه‌ی واحد و نقشه‌های تصادفی در جداول زیر آمده است.

جدول ۴-۵: نتایج شبیه‌سازی چهار الگوریتم روی نقشه 150×150

شاخص‌ها	الگوریتم A*	A* با فاصله توسعه	A* دوطرفه با فاصله توسعه	الگوریتم EBS-A*
زمان اجرا (ثانیه)	۲۶۷,۳۷۷	۱۰۰,۵۹۷	۵۶,۷۵۵	۵۶,۸۶۶
تعداد گره‌ها	۲۷۹	۲۸۵	۲۸۵	۲۲۸
تعداد گردش‌های ۹۰ درجه	۳۹	۴۷	۴۲	۰
حداکثر زاویه چرخش	۹۰°	۹۰°	۹۰°	۴۵°
تعداد گره‌های گسترش‌یافته	۷,۴۳۷	۲,۹۳۱	۲,۶۳۳	۲,۶۴۴
تعداد گره‌های بحرانی	۱۹۲	۰	۰	۱۱

جدول ۴-۶: میانگین نتایج شبیه‌سازی چهار الگوریتم روی نقشه تصادفی 150×150

شاخص‌ها	الگوریتم A*	A* با فاصله توسعه	A* دوطرفه با فاصله توسعه	الگوریتم EBS-A*
زمان اجرا (ثانیه)	۲۶۴,۷۸۹	۱۱۶,۳۷۹	۴۵,۳۸۱	۵۷,۰۳۲
تعداد گره‌ها	۲۷۹,۴	۲۸۴,۲	۲۸۳,۸	۲۳۳,۴
تعداد گردش‌های ۹۰ درجه	۲۹	۲۶,۲	۲۸	۰
حداکثر زاویه چرخش	۹۰°	۹۰°	۹۰°	۴۵°
تعداد گره‌های گسترش‌یافته	۸,۸۶۴,۴	۳,۷۱۳,۲	۲,۷۱۳	۳,۳۲۵,۶
تعداد گره‌های بحرانی	۲۰۱	۰	۰	۱۲

اما کارایی الگوریتم A* برابر با ۴,۷ برابر الگوریتم A* است. دلیل این امر این است که در نقشه 100×100 موانع کمی وجود دارد، اما در نقشه 150×150 موانع متراکم وجود دارند. در محیطی با موانع متراکم، کارایی الگوریتم بیشتر است.

فصل پنجم

نتیجه گیری

در این پژوهش، ابتدا مسئله‌ی مسیریابی و یافتن مسیر بهینه در محیط‌های مختلف مورد بررسی قرار گرفت و سپس الگوریتم‌های پایه و پرکاربرد در این حوزه، به‌ویژه الگوریتم A^* ، تحلیل شدند. با توجه به محدودیت‌های موجود در روش‌های کلاسیک، نسخه‌های بهبودیافته‌ای از این الگوریتم‌ها معرفی و ارزیابی شدند تا بتوان دقت، سرعت و کیفیت مسیرهای تولیدشده را افزایش داد.

در ادامه، با تمرکز بر ساختار و عملکرد الگوریتم‌های منتخب، رویکردهای مختلفی برای کاهش زمان اجرا و بهبود فرآیند جستجو پیشنهاد شد. نتایج شبیه‌سازی‌ها نشان داد که به‌کارگیری این روش‌ها می‌تواند در بسیاری از سناریوها موجب کاهش تعداد گره‌های بررسی‌شده و در نتیجه افزایش کارایی الگوریتم شود. همچنین مقایسه‌ی نتایج در اندازه‌های مختلف محیط نشان داد که عملکرد الگوریتم پیشنهادی در محیط‌های بزرگ‌تر نیز قابل قبول و مؤثر است.

از سوی دیگر، نتایج به‌دست‌آمده بیانگر آن است که استفاده از ساختارهای کمکی و تکنیک‌های بهبود جستجو می‌تواند تأثیر قابل توجهی در کاهش پیچیدگی عملیاتی داشته باشد. با این حال، همچنان چالش‌هایی مانند وابستگی به نوع محیط، موانع موجود و نحوه‌ی تنظیم پارامترها باقی می‌ماند که می‌تواند در تحقیقات آینده مورد توجه قرار گیرد.

در مجموع، نتایج این پژوهش نشان می‌دهد که الگوریتم‌های بهبودیافته‌ی مبتنی بر A^* می‌توانند گزینه‌ای مناسب برای حل مسائل مسیریابی در کاربردهای مختلف باشند. انتظار می‌رود در پژوهش‌های آتی، با توسعه‌ی بیشتر این روش‌ها و استفاده از داده‌های واقعی‌تر، کارایی و دقت آن‌ها بیش از پیش افزایش یابد.

کتاب نامه

- [1] Dudi, Tanishk, Singhal, Rahul, and Kumar, Rajesh. Shortest path evaluation with enhanced linear graph and dijkstra algorithm. In *2020 59th Annual Conference of the Society of Instrument and Control Engineers of Japan (SICE)*, pages 451–456. IEEE, 2020.
- [2] Guruji, Akshay Kumar, Agarwal, Himansh, and Parsediya, DK. Time-efficient a* algorithm for robot path planning. *Procedia Technology*, 23:144–149, 2016.
- [3] Hart, Peter E, Nilsson, Nils J, and Raphael, Bertram. A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.
- [4] Karur, Karthik, Sharma, Nitin, Dharmatti, Chinmay, and Siegel, Joshua E. A survey of path planning algorithms for mobile robots. *Vehicles*, 3(3):448–468, 2021.
- [5] Liu, Qinghe, Zhao, Lijun, Tan, Zhibin, and Chen, Wen. Global path planning for autonomous vehicles in off-road environment via an a-star algorithm. *International Journal of Vehicle Autonomous Systems*, 13(4):330–339, 2017.
- [6] Peng, Jiansheng, Huang, Yiyong, and Luo, Guan. Robot path planning based on improved a* algorithm. *Cybernetics and Information Technologies*, 15(2):171–180, 2015.

-
- [7] Qingji, Gao, Yongsheng, Yu, and Dandan, Hu. feasible path search and optimization based on an improved a* algorithm. *China Civil Aviation College Journal*, 23(4):42–44, 2005.
- [8] Singh, Yogang, Sharma, Sanjay, Sutton, Robert, Hatton, Daniel, and Khan, Asiya. A constrained a* approach towards optimal path planning for an unmanned surface vehicle in a maritime environment containing dynamic obstacles and ocean currents. *Ocean Engineering*, 169:187–201, 2018.
- [9] Song, Rui, Liu, Yuanchang, and Bucknall, Richard. Smoothed a* algorithm for practical unmanned surface vehicle path planning. *Applied Ocean Research*, 83:9–20, 2019.
- [10] Wang, Ziqiang, Hu, Xiaoguang, Li, Xiaoxiao, and Du, ZQ. Overview of global path planning algorithms for mobile robots. *Computer Science*, 48(10):19–29, 2021.
- [11] Yang, Joe-Ming, Tseng, Chien-Ming, and Tseng, PS. Path planning on satellite images for unmanned surface vehicles. *International Journal of Naval Architecture and Ocean Engineering*, 7(1):87–99, 2015.
- [12] Zhang, Xu, Cheng, CQ, Hao, XY, Li, JS, and Hu, P. A dynamic path planning algorithm for robots considering both global and local characteristics. *Journal of Geomatics Science and Technology*, 35(3):315–320, 2018.